

CV1: Profilová karta v Jetpack Compose - Teória

1 Cieľ cvičenia

V tomto cvičení sa naučíme, ako sa v Jetpack Compose skladá používateľské rozhranie z funkcií označených `@Composable`, ako funguje `@Preview` a ako sa používajú základné layouty `Row`, `Column` a `Box`. Výsledkom bude profilová karta s avatarom, textami, štatistikami a tlačidlami. Ako extra úlohu pridáme jednoduchú interaktivitu pre *Follow/Unfollow*.

2 @Composable

`@Composable` označuje funkciu, ktorá kreslí UI. Takáto funkcia:

- môže volať iné composable funkcie,
- môže prijímať parametre (napr. texty, čísla),
- môže používať `Modifier` na úpravu rozmerov, pozície a štýlu.

```
@Composable
fun HelloName(name: String) {
    Text(text = "Ahoj, $name")
}
```

3 @Preview

`@Preview` zobrazí composable v Android Studio bez spustenia aplikácie. Pri preview používame statické hodnoty,

```
@Preview(showBackground = true)
@Composable
fun HelloNamePreview() {
    HelloName(name = "Jana")
}
```

4 Importy

Pri práci s týmto cvičením sa najčastejšie objavia importy z týchto skupín:

- `androidx.compose.runtime.*` pre `@Composable`, `remember`, `state`
- `androidx.compose.foundation.layout.*` pre `Row`, `Column`, `Box`, `Spacer`, `padding`, `size`, `weight`
- `androidx.compose.foundation.*` a `androidx.compose.ui.*` pre `background`, `clip`, `Alignment`, `Modifier`
- `androidx.compose.material3.*` pre `Surface`, `Text`, `Button`, `OutlinedButton`, `MaterialTheme`, `ButtonDefaults`
- `androidx.compose.ui.tooling.preview.Preview` pre `preview`

5 Layouty: Row vs Column vs Box

5.1 Column

Column ukladá prvky zvisle pod seba:

- prvý potomok je hore,
- ďalší sa pridáva pod ním.

```
Column {  
    Text("Prvy riadok")  
    Text("Druhy riadok")  
}
```

Dôležité správanie:

- ak do Column vložíme druhý Text, budú pod sebou,
- ak chceme medzeru, použijeme `Spacer(height = ...)`.

5.2 Row

Row ukladá prvky vodorovne vedľa seba:

- prvý potomok je vľavo,
- ďalší sa pridáva doprava.

```
Row {  
    Text("Vľavo")  
    Text("Vpravo")  
}
```

Dôležité správanie:

- ak dáme do Row druhý Column, budú vedľa seba,
- šírka potomkov sa riadi ich obsahom alebo `weight()`.

5.3 Box

Box je kontajner, ktorý:

- vie obsah prekryť (*overlay*),
- vie obsah zarovnať (napr. do stredu alebo do rohu).

```
Box {  
    Text("Vzadu")  
    Text("Vpredu")  
}
```

V Boxe sa druhý prvok vykreslí nad prvým. To je dôvod, prečo Box použijeme na avatar + badge.

Pri hlavičke profilovej karty používame Column, pretože chceme avatar vľavo a texty vpravo. V textovej časti používame Row, pretože meno a status chceme mať pod sebou. Nebojte sa používať if/else; využijete ho napríklad pri farbe online badge.

```
val badgeColor = if (isOnline) Color(0xFF2ECC71) else Color(0xFFB0B0B0)

Box(
    modifier = Modifier
        .size(14.dp)
        .clip(CircleShape)
        .background(badgeColor)
)
```

6 Alignment (zarovnanie potomkov)

Alignment hovorí, kde sa majú potomkovia zarovnať:

- v Row zvyčajne riešime vertikálne zarovnanie (verticalAlignment),
- v Column riešime horizontálne zarovnanie (horizontalAlignment),
- v Boxe môžeme zarovnať obsah cez contentAlignment.

```
Row(verticalAlignment = Alignment.CenterVertically) {
    Text("Avatar")
    Text("Text vedľa")
}

Column(horizontalAlignment = Alignment.CenterHorizontally) {
    Text("Hore")
    Text("Dole")
}

Box(contentAlignment = Alignment.Center) {
    Text("V strede")
}
```

7 Arrangement (rozloženie priestoru medzi deťmi)

Arrangement rieši medzery a rozloženie potomkov v osi layoutu:

- v Row: horizontálne rozloženie (`horizontalArrangement`),
- v Column: vertikálne rozloženie (`verticalArrangement`).

```
@Composable
fun CardContentExample() {
    Column(
        modifier = Modifier.padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        Text("Header sekcia")
        Text("Statistiky sekcia")
        Text("Tlacidla sekcia")
    }
}
```

8 Modifier

Modifier upravuje vzhľad a správanie prvku. Najčastejšie použijeme:

- `fillMaxSize()`, `fillMaxWidth()`
- `padding(...)`
- `size(...)` a `width(...)`
- `background(...)`
- `clip(...)`
- `weight(...)`

```
Text(
    text = "Text",
    modifier = Modifier
        .clip(RoundedCornerShape(16.dp))
        .background(Color.LightGray)
        .padding(16.dp)
)
```

Je dôležité vedieť, že záleží na poradí operácií v `Modifier` (zľava doprava). Napríklad, ak chceme, aby bolo pozadie tiež orezané do tvaru, najprv použijeme `clip(...)` a až potom `background(...)`.

9 weight() a delenie priestoru

`weight()` je nástroj, ktorým vieme povedať, aby si prvky v `Row` alebo `Column` delili voľný priestor. V `Row` delí šírku, v `Column` delí výšku. Je dôležité, aby predok mal jasnú dostupnú šírku alebo výšku, inak `weight()` nemusí pôsobiť tak, ako sa očakáva.

```
@Composable
fun StatsRow(posts: String, followers: String, following: String)
{
    Row(
        modifier = Modifier.fillMaxWidth(),
        horizontalArrangement = Arrangement.spacedBy(8.dp)
    ) {
        StatItem(modifier = Modifier.weight(1f), value = posts,
            label = "Posts")
        StatItem(modifier = Modifier.weight(1f), value =
            followers, label = "Followers")
        StatItem(modifier = Modifier.weight(1f), value =
            following, label = "Following")
    }
}
```

10 Spustenie aplikácie

Jetpack Compose UI sa nezobrazí samo od seba. Android aplikácia sa spúšťa cez `Activity`. Táto `Activity` v metóde `onCreate()` zavolá `setContent { ... }`. V tomto bloku sa nachádza prvý composable, u nás `ProfileScreen()`. Ak ho nezavoláme, aplikácia môže bežať, ale obrazovka bude prázdna alebo sa zobrazí len základný obsah z iného šablónového kódu.

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MaterialTheme {
                Surface {
                    ProfileScreen()
                }
            }
        }
    }
}
```